z/OS System Initialization

TEKsystems Global Services

WORKFORCE DEVELOPMENT

PLATFORM ENGINEERING ROLE RE Z/OS

Cross-platform (UNIX-Linux-Windows-z/OS)

- System administration
- Software installation & upgrades
- System configuration
- Troubleshooting
- Automation
- Assistance to users/developers

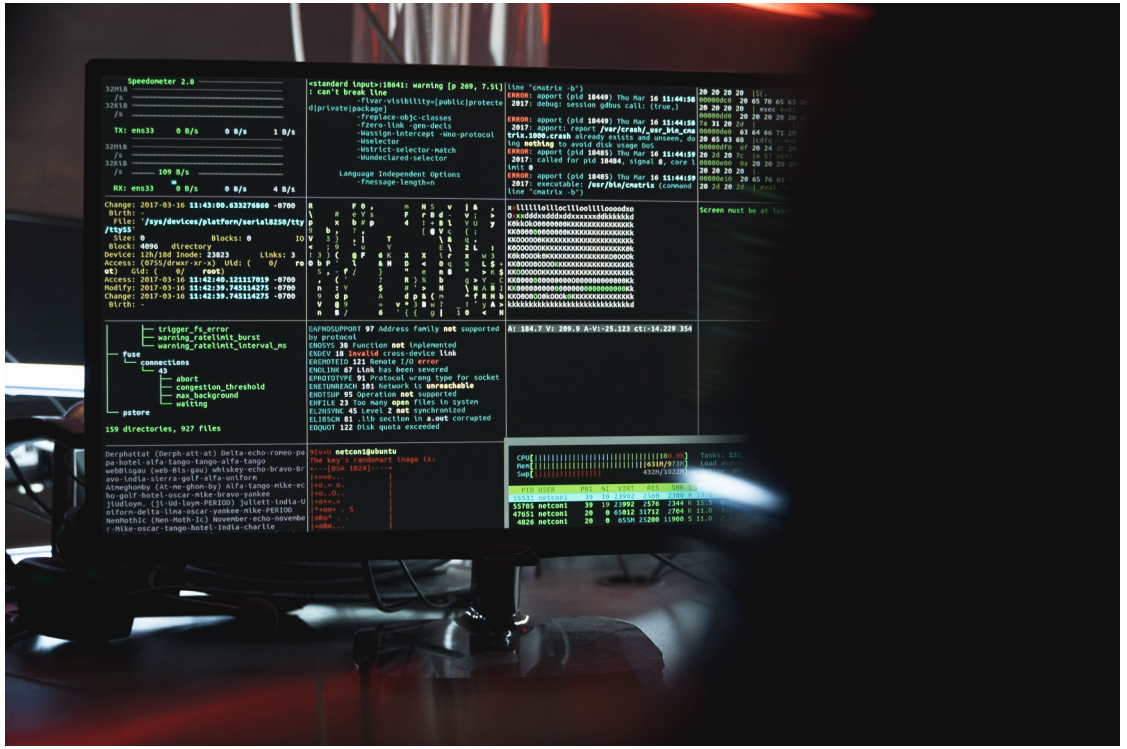
These responsibilities describe the job of a z/OS System Programmer. A Platform Engineer's job is more demanding because they must do these things across multiple operating systems, each of which has its own structure, commands and tools, procedures, and idiosyncratic behavior. You're (probably) already a system administrator for Linux/Unix and maybe Microsoft Windows®. Now you'll become one for z/OS, too.

Z/OS SYSTEM PROGRAMMING CONCEPTS

You can't point Ansible at a z/OS endpoint and run the same playbooks as you do to manage Unix, Linux, or Microsoft Windows®. You have to know a bit about how a z/OS system works.

Some of the key topics include:

- How z/OS initializes and loads configuration
- How system libraries and PARMLIB work
- How to design and maintain a z/OS environment
- How WLM, SMF, and JES2 operate
- RACF security fundamentals
- TCP/IP and z/OS UNIX System Services (USS) configuration



HIGH-LEVEL OS STRUCTURE

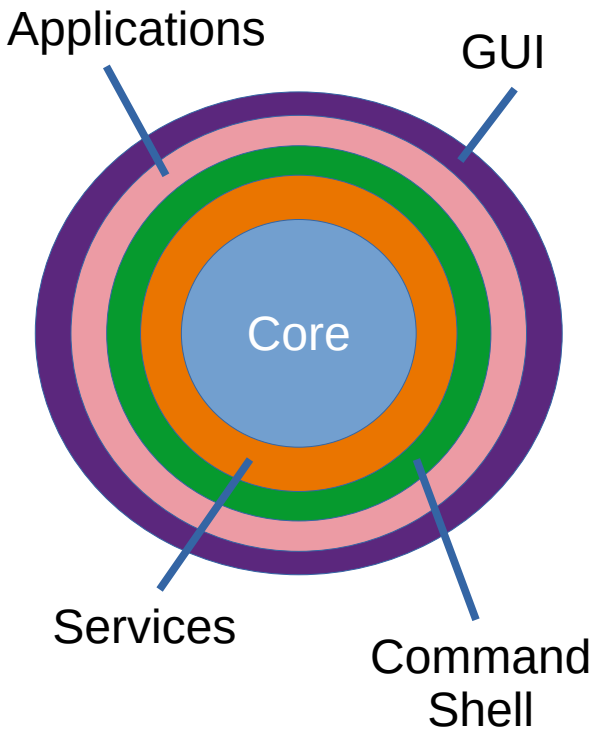
When computing was young, the machines didn't ship with operating systems or application software. People invented all kinds of ways to make them work, usually mixing different kinds of functionality in one program, including functions we associate with operating systems, device drivers, filesystems, and applications.

Some systems still work that way – certain embedded systems, high-performance graphic systems that have memory or CPU constraints, etc.

In time, general-purpose operating systems came to have a broadly-similar structure. Core OS functionality manages registers, address latching, condition codes, and exceptions, and handles task dispatching. It depends on add-on software, usually called *services*, to perform specialized work such as network communication, GUI management, or printing. Filesystems manage the details of persistent storage. Device drivers handle the details of interacting with peripheral devices. A command shell enables people to control (or believe they control) the OS. The OS can be tailored through configuration files/settings or by replacement of selected plug-and-play modules.

Although the details of z/OS are pretty different from those of more-familiar OSes like Unix, Linux, and Microsoft Windows®, at a certain level of abstraction it's quite similar to them. All these OSes start up by loading the core of the OS, which loads a service that handles loading all the other services, and so forth until the system is ready to perform useful work.

Let's see what that looks like on z/OS.



Highly authoritative diagram of operating system structure

Z/OS SYSTEM STARTUP RELATED WITH LINUX

Linux

boot (short for "bootstrap")

1. Firmware initialization

BIOS or UEFI performs power-on self test (POST), initializes hardware, sets bootable device order.

2. Boot device selection

Firmware selects/reads boot device (disk, network, USB) per settings.

3. Boot Loader Stage 1

MBR (legacy) or EFI system partition code runs; locates next-stage loader.

4. Boot Loader Stage 2 / kernel loader

GRUB/GRUB2, systemd-boot, or other boot manager presents menu, loads Linux kernel image and initramfs into memory.

z/OS

Initial Program Load (IPL)

1. IPL device selection

Operator or automated procedure selects "IPLable" device, or network/console command triggers IPL; device and load module specified.

2. Hardware IPL Start

Channel subsystem and microcode initialization. Machine placed in Initial State.

3. Initial Program is loaded

Load Program Supervisor is loaded from IPL device into memory via channel programs.

4. Load the z/OS Nucleus

IPL loads the "initial load module" or "z/OS nucleus", which contains the z/OS nucleus and control blocks. Data set SYS1.NUCLEUS.

Z/OS SYSTEM STARTUP RELATED WITH LINUX

Linux

boot (short for "bootstrap")

1. Firmware initialization

BIOS or UEFI performs power-on self test (POST), initializes hardware, sets bootable device order.

2. Boot device selection

Firmware selects/reads boot device (disk, network, USB)

3. Bo

MBR (
locate

4. Bo

GRUB
manager presents menu, loads Linux kernel image and initramfs into memory.

z/OS

Initial Program Load (IPL)

1. IPL device selection

Operator or automated procedure selects "IPLable" device, or network/console command triggers IPL; device and load module specified.

2. Microcode initialization

ode initialization.

ded from IPL device
ms.

e" or "z/OS
S nucleus and
control blocks. Data set SYS1.NUCLEUS.

- IPL process begins when the Operator uses the Hardware Management Console (HMC) and chooses the LOAD function.
- The HMC is an HTTP app that replaces what was (in the previous millennium) a physical console.
- Operator selects device to IPL from. Initial code can be on a SCSI or NVMe device.
- System reads IPL record, bootstrap, and IPL text into memory.

Z/OS SYSTEM STARTUP RELATED WITH LINUX

Linux

boot (short for "bootstrap

1. Firmware initializatio

BIOS or UEFI performs p
initializes hardware, sets

2. Boot device selectio

Firmw
USB)

3. Boo

MBR
locate

4. Boo

GRUB
manager presents menu
and initramfs into memory.

z/OS

Initial Program Load (IPL)

1. IPL device selection

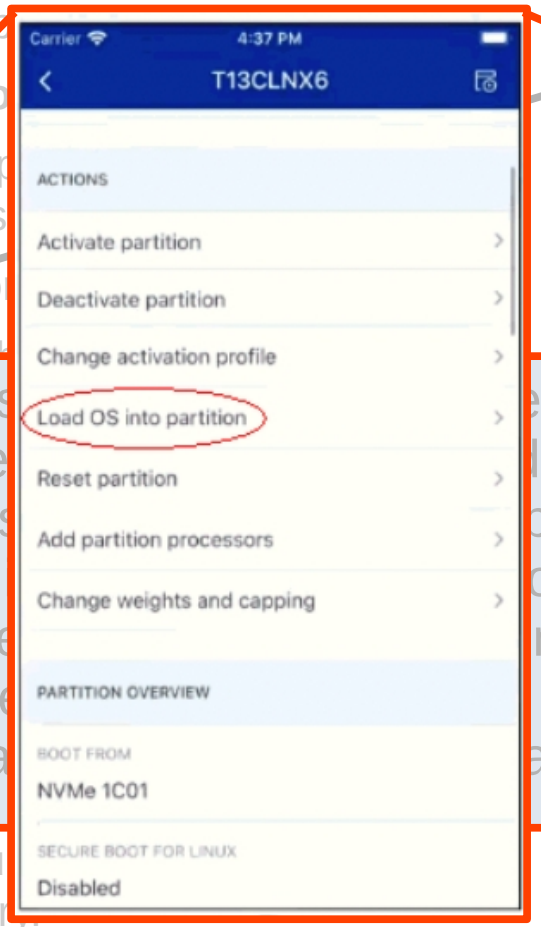
Operator or automated procedure selects "IPLable" device, or network/console command triggers IPL; device and load module specified.

2. Microcode initialization

ode initialization.

ded from IPL device
ms.

e" or "z/OS
S nucleus and
control blocks. Data set SYS1.NUCLEUS.



- IPL process
  Manageme
- The HMC is
  previous m
- Operator se
  or NVMe de
- System rea

erator uses the Hardware
l chooses the LOAD function.
places what was (in the
onsole.
m. Initial code can be on a SCSI
ap, and IPL text into memory.

Z/OS SYSTEM STARTUP RELATED WITH LINUX

Linux

boot (short for "bootstrap

1. Firmware initializatio

BIOS or UEFI performs p
initializes hardware, sets

2. Boot device selectio

Firmw
USB)

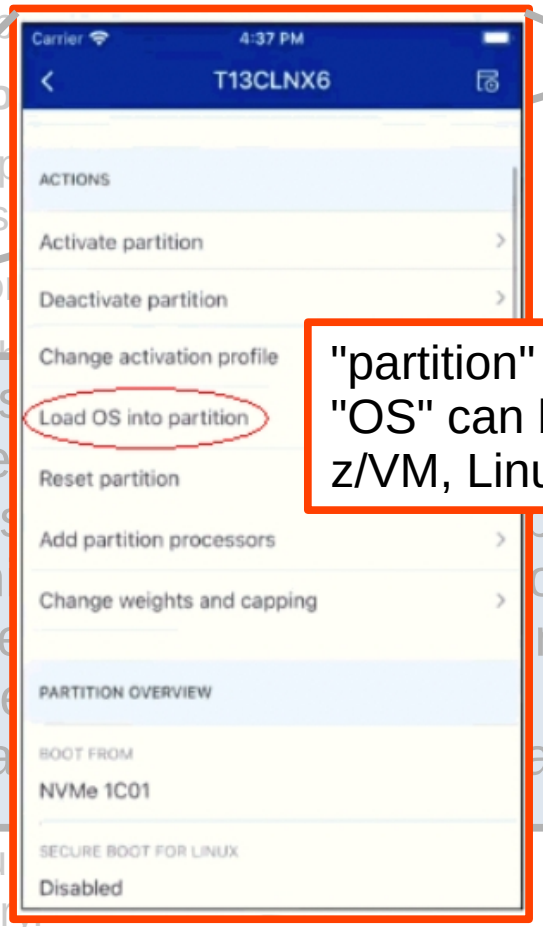
- IPL process
  Manageme
- The HMC is
  previous m
- Operator se
  or NVMe de
- System rea

3. Boo

MBR
locate

4. Boo

GRUB
manager presents menu
and initramfs into memory.



"partition" means LPAR
"OS" can be z/OS, z/VSE, z/TPF, z/VM, Linux, or KVM

z/OS

Initial Program Load (IPL)

1. IPL device selection

Operator or automated procedure selects "IPLable" device, or network/console command triggers IPL; device and load module specified.

2. Microcode initialization

ode initialization.
are
nction.
places what was (in the
onsole.
ded from IPL device
ms.
m. Initial code can be on a SCSI
ap, and IPL text into memory.
e" or "z/OS
S nucleus and
control blocks. Data set SYS1.NUCLEUS.

Z/OS SYSTEM STARTUP RELATED WITH LINUX

Linux

boot (short for "bootstrap")

1. Firmware initialization

BIOS or UEFI performs power-on self test (POST), initializes hardware, sets bootable device order.

2. Boot device selection

Firmware selects/reads boot device (disk, network, USB) per settings.

3. Boot Loader Stage 1

MBR (legacy) or EFI system partition code runs; locates next-stage loader.

4. Boot Loader Stage 2 / kernel loader

GRUB/GRUB2, systemd-boot, or other boot manager presents menu, loads Linux kernel image and initramfs into memory.

z/OS

Initial Program Load (IPL)

1. IPL device selection

Operator or automated procedure selects "IPLable" device, or network/console command triggers IPL; device and load module specified.

2. Hardware IPL Start

Channel subsystem and microcode initialization. Machine placed in Initial State.

3. Initial Program is loaded

Load Program Supervisor is loaded from IPL device into memory via channel programs.

4. Load the z/OS Nucleus

IPL loads the "initial load module" or "z/OS nucleus", which contains the z/OS nucleus and control blocks. Data set SYS1.NUCLEUS.

Z/OS SYSTEM STARTUP RELATED WITH LINUX

Linux

boot (short for "bootstrap")

1. Firmware initialization

BIOS or UEFI performs power-on self test (POST), initializes hardware, sets bootable device order.

2. Boot device selection

Firmware selects/reads boot device (disk, network, USB) per settings.

3. Boot Loader Stage 1

MBR (legacy) or EFI system partition code runs; locates next-stage loader.

4. Boot Loader Stage 2 / kernel loader

GRUB/GRUB2, systemd-boot, or other boot manager presents menu, loads Linux kernel image and initramfs into memory.

z/OS

Initial Program Load (IPL)

1. IPL device selection

Operator or automated procedure selects "IPLable" device, or network/console command triggers IPL; device and load module specified.

2. Hardware IPL Start

Channel subsystem and microcode initialization. Machine placed in Initial State.

3. Initial Program is loaded

Load Program Supervisor is loaded from IPL device into memory via channel programs.

4. Load the z/OS Nucleus

IPL loads the "initial load module" or "z/OS nucleus", which contains the z/OS nucleus and control blocks. Data set SYS1.NUCLEUS.

Z/OS SYSTEM STARTUP RELATED WITH LINUX

Linux

boot (short for "bootstrap")

1. Firmware initialization

BIOS or UEFI performs power-on self test (POST), initializes hardware, sets bootable device order.

2. Boot device selection

Firmware selects/reads boot device (disk, network, USB) per settings.

3. Boot Loader Stage 1

MBR (legacy) or EFI system partition code runs; locates next-stage loader.

4. Boot Loader Stage 2 / kernel loader

GRUB/GRUB2, systemd-boot, or other boot manager presents menu, loads Linux kernel image and initramfs into memory.

z/OS

Initial Program Load (IPL)

1. IPL device selection

Operator or automated procedure selects "IPLable" device, or network/console command triggers IPL; device and load module specified.

2. Hardware IPL Start

Channel subsystem and microcode initialization. Machine placed in Initial State.

3. Initial Program is loaded

Load Program Supervisor is loaded from IPL device into memory via channel programs.

4. Load the z/OS Nucleus

IPL loads the "initial load module" or "z/OS nucleus", which contains the z/OS nucleus and control blocks.

Z/OS SYSTEM STARTUP RELATED WITH LINUX

Linux

5. Kernel start

Linux kernel decompresses, initializes CPUs, memory management, device drivers; mounts initramfs as root.

6. Early init / hardware bring-up

Kernel probes and initializes devices, brings up drivers, starts udev/hotplug handling, switches from initramfs to real root filesystem.

7. System initialization / control blocks

Kernel starts userspace init (systemd, SysV init, or other – this is PID 1), runs init scripts or units to start services. Kernel sets up process table and scheduling.

z/OS

5. Nucleus start

z/OS nucleus initializes processor state, storage key settings, control blocks, and dispatcher environments. Two versions of nucleus are loaded: DAT-on (Dynamic Address Translation) and DAT-off (transient, for startup – real memory addressing).

6. Initialize I/O hardware

z/OS configures channels, channel paths, and I/O devices and establishes I/O subsystem (IOF/IOS structures).

7. Initialize control regions

z/OS initializes control regions (SRB, TCB), system control blocks (e.g., PSW, TCB/ASCB structures),

Z/OS SYSTEM STARTUP RELATED WITH LINUX

Linux

5. Kernel start

Linux kernel decompresses, initializes CPUs, memory management, device drivers; mounts initramfs as root.

z/OS

5. Nucleus start

z/OS nucleus initializes processor state, storage key settings, control blocks, and dispatcher environments. Two versions of nucleus are loaded: DAT-on (Dynamic Address Translation) and DAT-off (transient, for startup – real memory addressing).

- DAT-off nucleus
  - Basic
  - Before virtual memory is active
  - Loads base nucleus SYS1.NUCLEUS
- DAT-on nucleus
  - Full virtual memory enabled
- NUCMAP built
  - Map of nucleus in memory
- Similar to Linux
  - vmlinuz (Kernel) = DAT-off
  - initrd (RAM disk) = DAT-on

6. Initialize I/O hardware

z/OS configures channels, channel paths, and I/O devices and establishes I/O subsystem (IOF/IOS structures).

7. Initialize control regions

z/OS initializes control regions (SRB, TCB), system control blocks (e.g., PSW, TCB/ASCB structures),

Z/OS SYSTEM STARTUP RELATED WITH LINUX

Linux

5. Kernel start

Linux kernel decompresses, initializes CPUs, memory management, device d… initramfs as root.

z/OS

5. Nucleus start

z/OS nucleus initializes processor state, storage

- DAT-off nucleus
  - Basic
  - Before virtual mem
  - Loads base nucle
- DAT-on nucleus
  - Full virtual memor
- NUCMAP built
  - Map of nucleus in
- Similar to Linux
  - vmlinuz (Kernel) = DAT-off
  - initrd (RAM disk) = DAT-on

Guides the IPL loader when placing the nucleus image read from the IPL volume into real storage (or into target virtual addresses) so that control blocks and entry points are at expected offsets.

Supplies the nucleus with the information needed to establish translation (DAT) boundaries, set up storage keys, and initialize the translation tables/ TLBs when enabling DAT.

Helps support multiple nucleus variants or overlays (serviceable/permissive / diagnostic) by specifying alternative placements and which segments to load.

Enables the nucleus to locate important control-blocks (PSW, CCW/ IO control areas, dispatcher areas) immediately after load before full OS address-space structures are built.

Z/OS SYSTEM STARTUP RELATED WITH LINUX

Linux

5. Kernel start

Linux kernel decompresses, initializes CPUs, memory management, device drivers; mounts initramfs as root.

6. Early init / hardware bring-up

Kernel probes and initializes devices, brings up drivers, starts udev/hotplug handling, switches from initramfs to real root filesystem.

7. System initialization / control blocks

Kernel starts userspace init (systemd, SysV init, or other – this is PID 1), runs init scripts or units to start services. Kernel sets up process table and scheduling.

z/OS

5. Nucleus start

z/OS nucleus initializes processor state, storage key settings, control blocks, and dispatcher environments. Two versions of nucleus are loaded: DAT-on (Dynamic Address Translation) and DAT-off (transient, for startup – real memory addressing).

6. Initialize I/O hardware

z/OS configures channels, channel paths, and I/O devices and establishes I/O subsystem (IOF/IOS structures).

7. Initialize control regions

z/OS initializes control regions (SRB, TCB), system control blocks (e.g., PSW, TCB/ASCB structures),

Z/OS SYSTEM STARTUP RELATED WITH LINUX

Linux

5. Kernel start

Linux kernel decompresses, initializes CPUs, memory management, device drivers; mounts initramfs as root.

6. Early init / hardware bring-up

Kernel probes and initializes devices, brings up drivers, starts udev/hotplug handling, switches from initramfs to real root filesystem.

7. System initialization / control blocks

Kernel starts userspace init (systemd, SysV init, or other – this is PID 1), runs init scripts or units to start services. Kernel sets up process table and scheduling.

z/OS

5. Nucleus start

z/OS nucleus initializes processor state, storage key settings, control blocks, and dispatcher environments. Two versions of nucleus are loaded: DAT-on (Dynamic Address Translation) and DAT-off (transient, for startup – real memory addressing).

6. Initialize I/O hardware

z/OS configures channels, channel paths, and I/O devices and establishes I/O subsystem (IOF/IOS structures).

7. Initialize control regions

z/OS initializes control regions (SRB, TCB), system control blocks (e.g., PSW, TCB/ASCB structures).

Z/OS SYSTEM STARTUP RELATED WITH LINUX

Linu

5. Ke

Linux
mem
initra

6. Ea

Kern
drive
initra

7. Sy

Kern
other
start services. Kernel sets up process table and scheduling.

z/OS

5. Nucleus start

z/OS nucleus initializes processor state, storage key settings, control blocks, and dispatcher environments. Two versions of nucleus are loaded: DAT-on (Dynamic Address Translation) and DAT-off (transient, for startup – real memory addressing).

6. Initialize I/O hardware

z/OS configures channels, channel paths, and I/O devices and establishes I/O subsystem (IOF/IOS structures).

7. Initialize control regions

z/OS initializes control regions (SRB, TCB), system control blocks (e.g., PSW, TCB/ASCB structures).

Z/OS SYSTEM STARTUP RELATED WITH LINUX

PSW – Program Status Word

Control register containing the current instruction address (instruction counter), condition codes, interrupt masks, addressing mode bits, and control flags. Define's the processor's execution state. Saving/restoring the PSW performs context switches and returns from interrupts.

TCB – Task Control Block

Data structure representing a single task/thread. Holds a copy of the task's PSW and registers, scheduling info, priority, and pointers to its address space and resources. Used for dispatching, context switching, and accounting.

SRB – Service Request Block

Lightweight control block for short, high-priority work units; sort of a minimized TCB. Used for I/O completion, cross-address-space requests, and similar.

ASCB – Address Space Control Block

Describes an address space's page tables/translation structures, protection keys, resource limits, and pointers to dispatchable entities.

storage
her
are loaded:
and DAT-off
dressing).

s, and I/O
(IOF/IOS

CB), system
control blocks (e.g., PSW, TCB/ASCB structures).

other – this is PID 1), runs init scripts or units to start services. Kernel sets up process table and scheduling.

Z/OS SYSTEM STARTUP RELATED WITH LINUX

Linux

8. Mount filesystems

VFS mounts root and other filesystems (fstab, systemd mount units); network filesystems may be mounted.

9. Start system services

PID 1 process starts other services (e.g., networking, sshd, logging, DBs; multi-user targets reached.

10. Ready for logins

Login prompts provided on local console/tty or via network (ssh); system is available to users.

z/OS

8. Attach data sets

z/OS mounts/opens data sets and catalogs; Access Method Services (AMS) and catalog management make data available. Nucleus Initialization Program (NIP) reads configuration and finishes setup.

9. Start tasks

Master Scheduler starts started tasks and subsystems; primary subsystem first (JES); initiates workload managers and system logger.

10. Ready for sign on

Operator console displays messages; system is available to operators and users; users can submit jobs via JES or connect via consoles & terminals.

Z/OS SYSTEM STARTUP RELATED WITH LINUX

Linux

8. Mount filesystems

VFS mounts root and other filesystems (fstab, systemd mount units); network filesystems may be mounted.

9. Start system services

PID 1 process starts other services (e.g., networking, sshd, logging, DBs; multi-user targets reached.

10. Ready for logins

Login prompts provided on local console/tty or via network (ssh); system is available to users.

z/OS

8. Attach data sets

z/OS mounts/opens data sets and catalogs; Access Method Services (AMS) and catalog management make data available. Nucleus Initialization Program (NIP) reads configuration and finishes setup.

9. Start tasks

Master Scheduler starts started tasks and subsystems; primary subsystem first (JES); initiates workload managers and system logger.

10. Ready for sign on

Operator console displays messages; system is available to operators and users; users can submit jobs via JES or connect via consoles & terminals.

```
 BROWSE    SYS1.PROCLIB(TCPIP)                          Line 0000000000 Col 001 080
 Command ===>                                                  Scroll ===> CSR
******************************** Top of Data ********************************
//TCPIP    PROC PARMS='CTRACE(CTIEZB00)'
//S1       EXEC PGM=IKJEFT01,DYNAMNBR=75,TIME=1440,
//   PARM='RACFCRT VPCVSIIPADDRESS VSICA CERTEXPIRE'
//SYSPRINT DD SYSOUT=*
//SYSTSPRT DD SYSOUT=*
//SYSTERM  DD DUMMY
//SYSEXEC  DD DSN=SYS1.VS01.TSO.CLIST,DISP=SHR
//SYSTSIN  DD *
//*  Begin Ansible Block Insert  *//
//FEKJCL   EXEC PGM=IKJEFT01,DYNAMNBR=75,REGION=4M,
//   PARM='CXXPTRL'
//SYSPRINT DD SYSOUT=*
//SYSTSPRT DD SYSOUT=*
//SYSTERM  DD DUMMY
//SYSEXEC  DD DSN=SYS1.VS01.TSO.CLIST,DISP=SHR
//SYSTSIN  DD *
//FEKGSK  EXEC PGM=BPXBATCH,TIME=1440,COND=(8,GT,FEKJCL),
// PARM='sh /usr/lpp/IBM/zexpl/bin/fek.csfpgsk.rex -NAME=JWTTOK.FEKAPPL
//              -SIZE=256 -TYPE=CLEAR'
//STDOUT DD SYSOUT=*
//*
//IDZKEYS EXEC PGM=BPXBATCH,TIME=1440,
// PARM='sh /etc/idzkeys.sh'
//STDOUT DD SYSOUT=*
//*  End Ansible Block Insert  *//
//TCPIP    EXEC PGM=EZBTCPIP,PARM='&PARMS',REGION=0M,TIME=1440
//SYSPRINT DD  SYSOUT=*
//SYSERR   DD  SYSOUT=*
//SYSERROR DD  SYSOUT=*
//ERRORFIL DD  SYSOUT=*
//SYSDEBUG DD  SYSOUT=*
//PROFILE  DD  DISP=SHR,DSN=TCPIP.TCPPARMS(PROFILE)
//SYSTCPD  DD  DISP=SHR,DSN=TCPIP.TCPPARMS(TCPDATA)
******************************** Bottom of Data ********************************
```

LATED WITH LINUX

OS

Attach data sets

OS mounts/opens data sets and catalogs; Access ethod Services (AMS) and catalog management ake data available. Nucleus Initialization Program IP) reads configuration and finishes setup.

Start tasks

aster Scheduler starts started tasks and bsystems; primary subsystem first (JES); initiates orkload managers and system logger.

. Ready for sign on

perator console displays messages; system is ailable to operators and users; users can submit s via JES or connect via consoles & terminals.

LATED WITH LINUX

OS

Attach data sets

DS mounts/opens data sets and catalogs; Access
ethod Services (AMS) and catalog management
ake data available. Nucleus Initialization Program
IP) reads configuration and finishes setup.

Start tasks

aster Scheduler starts started tasks and
bsystems; primary subsystem first (JES); initiates
rkload managers and system logger.

Ready for sign on

or console displays messages; system is
le to operators and users; users can submit
JES or connect via consoles & terminals.

```
 BROWSE    SYS1.PROCLIB(TCPIP)                    Line 0000000000 Col 001 080
 Command ===>                                                 Scroll ===> CSR
******************************** Top of Data ********************************
//TCPIP    PROC PARMS='CTRACE(CTIEZB00)'
//S1       EXEC PGM=IKJEFT01,DYNAMNBR=75,TIME=1440,
//   PARM='RACFCRT VPCVSIIPADDRESS VSICA CERTEXPIRE'
//SYSPRINT DD SYSOUT=*
//SYSTSPRT DD SYSOUT=*
//SYSTERM  DD DUMMY
//SYSEXEC  DD DSN=SYS1.VS01.TSO.CLIST,DISP=SHR
//SYSTSIN  DD *
//*  Begin Ansible Block Insert  *//
//FEKJCL   EXEC PGM=IKJEFT01,DYNAMNBR=75,REGION=4M,
//   PARM='CXXPTRL'
//SYSPRINT DD SYSOUT=*
//SYSTSPRT DD SYSOUT=*
//SYSTERM  DD DUMMY
//SYSEXEC  DD DSN=SYS1.VS01.TSO.CLIST,DISP=SHR
//SYSTSIN  DD *
//FEKGSK  EXEC PGM=BPXBATCH,TIME=1440,COND=(8,GT,FEKJCL),
// PARM='sh /usr/lpp/IBM/zexpl/bin/fek.csfpgsk.rex -NAME=JWTTOK.FEKAPPL
//              -SIZE=256 -TYPE=CLEAR'
//STDOUT DD SYSOUT=*
//*
//IDZKEY
// PARM=
//STDOUT
//*  End
//TCPIP
//SYSPRINT DD  SYSOUT=*
//SYSERR   DD  SYSOUT=*
//SYSERROR DD  SYSOUT=*
//ERRORFIL DD  SYSOUT=*
//SYSDEBUG DD  SYSOUT=*
//PROFILE  DD  DISP=SHR,DSN=TCPIP.TCPPARMS(PROFILE)
//SYSTCPD  DD  DISP=SHR,DSN=TCPIP.TCPPARMS(TCPDATA)
******************************** Bottom of Data ********************************
```

This step runs TSO in batch mode to execute a REXX script in SYS1.VS01.TSO.CLIST(CXXPTRL) to check for a JWT token.

LATED WITH LINUX

OS

Attach data sets

DS mounts/opens data sets and catalogs; Access
ethod Services (AMS) and catalog management
ake data available. Nucleus Initialization Program
IP) reads configuration and finishes setup.

Start tasks

aster Scheduler starts started tasks and
bsystems; primary subsystem first (JES); initiates
rkload managers and system logger.

. Ready for sign on

perator console displays messages; system is
ailable to operators and users; users can submit
JES or connect via consoles & terminals.

```
 BROWSE    SYS1.PROCLIB(TCPIP)                   Line 0000000000 Col 001 080
 Command ===>                                              Scroll ===> CSR
******************************** Top of Data ********************************
//TCPIP    PROC PARMS='CTRACE(CTIEZB00)'
//S1       EXEC PGM=IKJEFT01,DYNAMNBR=75,TIME=1440,
//   PARM='RACFCRT VPCVSIIPADDRESS VSICA CERTEXPIRE'
//SYSPRINT DD SYSOUT=*
//SYSTSPRT DD SYSOUT=*
//SYSTERM  DD DUMMY
//SYSEXEC  DD DSN=SYS1.VS01.TSO.CLIST,DISP=SHR
//SYSTSIN  DD *
//*  Begin Ansible Block Insert  *//
//FEKJCL   EXEC PGM=IKJEFT01,DYNAMNBR=75,REGION=4M,
//   PARM='CXXPTRL'
//SYSPRINT DD SYSOUT=*
//SYSTSPRT DD SYSOUT=*
//SYSTERM  DD DUMMY
//SYSEXEC  DD DSN=SYS1.VS01.TSO.CLIST,DISP=SHR
//SYSTSIN  DD *
//FEKGSK  EXEC PGM=BPXBATCH,TIME=1440,COND=(8,GT,FEKJCL),
// PARM='sh /usr/lpp/IBM/zexpl/bin/fek.csfpgsk.rex -NAME=JWTTOK.FEKAPPL
//              -SIZE=256 -TYPE=CLEAR'
//STDOUT DD SYSOUT=*
//*
//IDZKEYS EXEC PGM=BPXBATCH,TIME=1440,
// PARM='sh /etc/idzkeys.sh'
//STDOUT DD SYSOUT=*
//*
//TC
//SY
//SY
//SYSERROR DD  SYSOUT=*
//ERRORFIL DD  SYSOUT=*
//SYSDEBUG DD  SYSOUT=*
//PROFILE  DD  DISP=SHR,DSN=TCPIP.TCPPARMS(PROFILE)
//SYSTCPD  DD  DISP=SHR,DSN=TCPIP.TCPPARMS(TCPDATA)
******************************** Bottom of Data ********************************
```

This step runs a USS shell script in MVS batch mode to generate a private key for token JWTTOK.FEKAPPL

LATED WITH LINUX

OS

Attach data sets

DS mounts/opens data sets and catalogs; Access
ethod Services (AMS) and catalog management
ake data available. Nucleus Initialization Program
IP) reads configuration and finishes setup.

Start tasks

aster Scheduler starts started tasks and
bsystems; primary subsystem first (JES); initiates
rkload managers and system logger.

. Ready for sign on

perator console displays messages; system is
ailable to operators and users; users can submit
s via JES or connect via consoles & terminals.

```
 BROWSE    SYS1.PROCLIB(TCPIP)                   Line 0000000000 Col 001 080
 Command ===>                                              Scroll ===> CSR
******************************** Top of Data ********************************
//TCPIP    PROC PARMS='CTRACE(CTIEZB00)'
//S1       EXEC PGM=IKJEFT01,DYNAMNBR=75,TIME=1440,
//   PARM='RACFCRT VPCVSIIPADDRESS VSICA CERTEXPIRE'
//SYSPRINT DD SYSOUT=*
//SYSTSPRT DD SYSOUT=*
//SYSTERM  DD DUMMY
//SYSEXEC  DD DSN=SYS1.VS01.TSO.CLIST,DISP=SHR
//SYSTSIN  DD *
//*  Begin Ansible Block Insert  *//
//FEKJCL   EXEC PGM=IKJEFT01,DYNAMNBR=75,REGION=4M,
//   PARM='CXXPTRL'
//SYSPRINT DD SYSOUT=*
//SYSTSPRT DD SYSOUT=*
//SYSTERM  DD DUMMY
//SYSEXEC  DD DSN=SYS1.VS01.TSO.CLIST,DISP=SHR
//SYSTSIN  DD *
//FEKGSK  EXEC PGM=BPXBATCH,TIME=1440,COND=(8,GT,FEKJCL),
// PARM='sh /usr/lpp/IBM/zexpl/bin/fek.csfpgsk.rex -NAME=JWTTOK.FEKAPPL
//              -SIZE=256 -TYPE=CLEAR'
//STDOUT DD SYSOUT=*
//*
//IDZKEYS EXEC PGM=BPXBATCH,TIME=1440,
// PARM='sh /etc/idzkeys.sh'
//STDOUT DD SYSOUT=*
//*  End Ansible Block Insert  *//
//TCPIP    EXEC PGM=EZBTCPIP,PARM='&PARMS',REGION=0M,TIME=1440
//SYSPRINT DD  SYSOUT=*
//SY
//SY
//EF
//SY
//PROFILE  DD  DISP=SHR,DSN=TCPIP.TCPPARMS(PROFILE)
//SYSTCPD  DD  DISP=SHR,DSN=TCPIP.TCPPARMS(TCPDATA)
******************************** Bottom of Data ********************************
```

This step runs a USS shell script in MVS batch mode to create the key store if it doesn't already exist.

Z/OS SYSTEM STARTUP RELATED WITH LINUX

Linux

8. Mount filesystems

VFS mounts root and other filesystems (fstab, systemd mount units); network filesystems may be mounted.

9. Start system services

PID 1 process starts other services (e.g., networking, sshd, logging, DBs; multi-user targets reached.

10. Ready for logins

Login prompts provided on local console/tty or via network (ssh); system is available to users.

z/OS

8. Attach data sets

z/OS mounts/opens data sets and catalogs; Access Method Services (AMS) and catalog management make data available. Nucleus Initialization Program (NIP) reads configuration and finishes setup.

9. Start tasks

Master Scheduler starts started tasks and subsystems; primary subsystem first (JES); initiates workload managers and system logger.

10. Ready for sign on

Operator console displays messages; system is available to operators and users; users can submit jobs via JES or connect via consoles & terminals.

MASTER SCHEDULER

On Linux systems that run systemd, process id 1 (PID 1) is the systemd service. It's the ancestor of all the other services on the system.

The conceptual equivalent on z/OS is the Master Scheduler, which is started during IPL and stays active while z/OS is running.

Address spaces on z/OS have Address Space Identifiers (ASIDs), which you can think of as somewhat analogous to PIDs. The Master Scheduler address space is ASID 1.

Technically the ASIDs are not PIDs because they don't identify specific running processes. This is only a conceptual analogy.

THREE KINDS OF IPL

Cold IPL

Complete initialization from scratch

Warm IPL

Partial re-use of previous state – typical

Quick IPL

Minimal initialization

IPL-ING THE LAB SYSTEM

We will not IPL the Lab system, but if we did then we would use the LOAD function of the HMC to do so. It issues an operator START command.

To start a z/OS instance with the installation default configuration:

START

- or -

S D M_START

To start the Lab environment when it isn't the installation default (all one line):

```
START  LOAD00=(K2.PARMLIB,USRVS1),        <= PARMLIB concatenation
       LOAD01=SYS1.PARMLIB,
       LOAD02=SYS1.PARMLIB.INSTALL,
       IPL=Z31VS1                         <= location of SYS1.NUCLEUS
```

IPL-ING THE LAB SYSTEM

SYS0.IPLPARM(LOADK2)

K2.PARMLI

```
VIEW       SYS0.IPLPARM(LOADK2) - 01.00                    Columns 00
Command ===>                                                 Scroll
****** ***************************** Top of Data ********************
000001 *---+----1----+----2----+----3----+----4----+----5----+----6--
000002 IODF     00 PROV     DEFAULT  00
000003 INITSQA        0001M
000004 SYSCAT   OPEVS1143CCATALOG.VS01.MASTER
000005 IEASYM   (00,K2)
000006 SYSPLEX  LOCAL
000007 PARMLIB  K2.PARMLIB                                  USRVS1
000008 PARMLIB  SYS1.PARMLIB
000009 PARMLIB  SYS1.PARMLIB.INSTALL
****** **************************** Bottom of Data ******************
```

(MSTJCL00)

IPL-ING THE LAB SYSTEM

SYS0.IPLPARM(LOADK2)

K2.PARMLI

```
VIEW       SYS0.IPLPARM(LOADK2) - 01.00                          Columns 00
Command ===>                                                      Scroll
****** ******************************* Top of Data ********************
000001 *---+----1----+----2----+----3----+----4----+----5----+----6--
000002 IODF     00 PROV     DEFAULT  00
000003 INITSQA        0001M
000004 SYSCAT   OPEVS1143CCATALOG.VS01.MASTER
000005 IEASYM   (00,K2)
000006 SYSPLEX  LOCAL
000007 PARMLIB  K2.PARMLIB                                    USRVS1
000008 PARMLIB  SYS1.PARMLIB
                                                  m of Data ***********
```

LOADxx members in IPLPARM are system configuration data sets.
The gory details are in the IBM z/OS documentation.

- LOADK2 specifies system configuration for our lab environment.
- IODF points to a hardware I/O configuration
- INITSQA specifies 1MB for the initial System Queue Area, just for startup
- SYSCAT identifies the system catalog to be used for this system
- IEASM specifies the suffix(es) of a PARMLIB member or members containing SYSDEF and/or SYMDEF definitions
- SYSPLEX name defaults to LOCAL; explicitly given in this case
- PARMLIB concatenation for startup

IPL-ING THE LAB SYSTEM

SYS0.IPLPARM(LOADK2)

K2.PARMLIB(IEASYMK2)

(JES2PARM)

(MSTJCL00)

```
VIEW       K2.PARMLIB(IEASYMK2) - 01.03
Command ===>
****** ***************************** Top of Data ******
000001 SYSDEF
000002   SYMDEF(&HOMEIPADDRESS1.='173.45.66.189')
000003   SYMDEF(&DEFAULTROUTEADDR.='192.168.200.229')
000004   SYMDEF(&OSAPORTNAME.='osafe')
000005   SYMDEF(&TCPHOSTNAME_.='ZSYSTECH')
000006   SYMDEF(&NAMESERVERADDR1.='8.8.8.8')
000007   SYMDEF(&NAMESERVERADDR2.='8.8.4.4')
000008   SYMDEF(&NAMESERVERPORT.='53')
000009   SYMDEF(&VPCVSIIPADDRESS.='192.168.200.229')
000010   SYMDEF(&TCPDOMAIN_.='')
****** **************************** Bottom of Data ****
```

IPL-ING THE LAB SYSTEM

SYS0.IPLPARM(LOADK2)

K2.PARMLIB(IEASYMK2)

(JES2PARM)

```
VIEW       K2.PARMLIB(IEASYMK2) - 01.03
Command ===>
****** ***************************** Top of Data ******
000001 SYSDEF
000002   SYMDEF(&HOMEIPADDRESS1.='173.45.66.189')
000003   SYMDEF(&DEFAULTROUTEADDR.='192.168.200.229')
000004   SYMDEF(&OSAPORTNAME.='osafe')
000005   SYMDEF(&TCPHOSTNAME_.='ZSYSTECH')
000006   SYMDEF(&NAMESERVERADDR1.='8.8.8.8')
000007   SYMDEF(&NAMESERVERADDR2.='8.8.4.4')
000008   SYMDEF(&NAMESERVERPORT.='53')
000009   SYMDEF(&VPCVSIIPADDRESS.='192.168.200.229')
000010   SYMDEF(&TCPDOMAIN_.='')
********************** Bottom of Data ****
```

The first IEASYMxx member found is member IEASYMK2 in K2.PARMLIB. The first one found is used. The member assigns values to symbols that are used later in the IPL process.

The keyword SYSDEF is also coded in this member, in this case. SYSDEF can be given in a member named IEASYSxx. That is not used here. This SYSDEF doesn't override any installation defaults. It still has to be present somewhere, even if it has no parameters.

IPL-ING THE LAB SYSTEM

SYS0.IPLPARM(LOADK2)

K2.PARMLIB(IEASYMK2)

(JES2PARM)

(MSTJCL00)

(partial)

```
VIEW       K2.PARMLIB(JES2PARM) - 01.00                 Columns 00001 000
Command ===> _____________________________________________ Scroll ===> CS
000086 /*********************************************************************
000087 *  JOBCLASS Definitions
000088 *********************************************************************/
000089 JOBCLASS(A) COMMAND=DISPLAY
000090
000091 /*********************************************************************
000092 *  Dynamic PROCLIB Concatenation
000093 *********************************************************************/
000094 PROCLIB(PROC00) DD(1)=(DSN=K2.PROCLIB),
000095                 DD(2)=(DSN=SYS1.PROCLIB,UNIT=SYSALLDA,
000096                        VOLSER=OPEVS1),
000097                 DD(3)=(DSN=SYS1.PROCLIB.INSTALL,UNIT=SYSALLDA,
000098                        VOLSER=&SYSR1.)
000099 PROCLIB(PROC01) DD(1)=(DSN=K2.PROCLIB),
000100                 DD(2)=(DSN=SYS1.PROCLIB.INSTALL,UNIT=SYSALLDA,
000101                        VOLSER=&SYSR1.)
000102
000103 /*********************************************************************
000104 *  Displays, MVS commands (COMMNDx too early), JES2 auto-cmds.
000105 *********************************************************************/
000106
000107 DISPLAY SPOOLDEF DSNAME,VOLUME
000108 DISPLAY CKPTDEF  CKPT1
000109 $D JOBDEF
000110 $D MR0,'EHK602I : JES2 IS NOW ACCEPTING COMMANDS'
000111 $VS,'WRITELOG START'
000112 /*  Begin Ansible insert 1 */
000113 INITDEF PARTNUM=10
000114 OUTDEF JOENUM=9999
000115 JOBDEF JOBNUM=3000
000116 /*  End Ansible insert 1 */
****** ***************************** Bottom of Data ************************
```

IPL-ING THE LAB SYSTEM

SYS0.IPLPARM(LOADK2)

K2.PARMLIB(IEASYMK2)

(JES2PARM)

The JES2PARM member in PARMLIB contains configuration settings for JES2, the Job Entry Subsystem. This example shows only a portion of the member.

(partial)

```
VIEW       K2.PARMLIB(JES2PARM) - 01.00                  Columns 00001 000
Command ===> _____________________________________________ Scroll ===> CS
000086 /*********************************************************************
000087 *  JOBCLASS Definitions
000088 *********************************************************************/
000089 JOBCLASS(A) COMMAND=DISPLAY
000090
000091 /*********************************************************************
000092 *  Dynamic PROCLIB Concatenation
000093 *********************************************************************/
000094 PROCLIB(PROC00) DD(1)=(DSN=K2.PROCLIB),
000095                 DD(2)=(DSN=SYS1.PROCLIB,UNIT=SYSALLDA,
000096                        VOLSER=OPEVS1),
000097                 DD(3)=(DSN=SYS1.PROCLIB.INSTALL,UNIT=SYSALLDA,
000098                        VOLSER=&SYSR1.)
000099 PROCLIB(PROC01) DD(1)=(DSN=K2.PROCLIB),
000100                 DD(2)=(DSN=SYS1.PROCLIB.INSTALL,UNIT=SYSALLDA,
000101                        VOLSER=&SYSR1.)
000102
000103 /*********************************************************************
000104 *  Displays, MVS commands (COMMNDx too early), JES2 auto-cmds.
000105 *********************************************************************/
000106
000107 DISPLAY SPOOLDEF DSNAME,VOLUME
000108 DISPLAY CKPTDEF  CKPT1
000109 $D JOBDEF
000110 $D MR0,'EHK602I : JES2 IS NOW ACCEPTING COMMANDS'
000111 $VS,'WRITELOG START'
000112 /*  Begin Ansible insert 1 */
000113 INITDEF PARTNUM=10
000114 OUTDEF JOENUM=9999
000115 JOBDEF JOBNUM=3000
000116 /*  End Ansible insert 1 */
****** ***************************** Bottom of Data ************************
```

IPL-ING THE LAB SYSTEM

SYS0.IPLPARM(LOADK2)

K2.PARMLIB(IEASYMK2)

(JES2PARM)

(MSTJCL00)

```
VIEW       K2.PARMLIB(MSTJCL00) - 01.00
Command ===>
****** ***************************** Top of Data **
000001 //MSTJCL00 JOB  MSGLEVEL=(1,1),TIME=1440
000002 //          EXEC PGM=IEEMB860,DPRTY=(15,15)
000003 //STCINRDR DD  SYSOUT=(A,INTRDR)
000004 //TSOINRDR DD  SYSOUT=(A,INTRDR)
000005 //IEFPDSI  DD DSN=K2.PROCLIB,DISP=SHR,
000006 //             UNIT=SYSALLDA,VOL=SER=USRVS1
000007 //          DD  DSN=SYS1.PROCLIB,DISP=SHR
000008 //IEFPARM  DD  DSN=SYS1.PARMLIB,DISP=SHR
000009 //SYSUADS  DD DSN=SYS1.VS01.UADS,
000010 //             DISP=SHR
000011 //SYSLBC   DD DSN=SYS1.VS01.BRODCAST,
000012 //             DISP=SHR
****** ***************************** Bottom of Data
```

IPL-ING THE LAB SYSTEM

SYS0.IPLPARM(LOADK2)

The MSTJCL00 member in PARMLIB is the JCL to start the Master Scheduler. Program name is IEEMB860 in this case. Multiple Master Schedulers can be written to enforce different installation policies, if necessary.

The Master Scheduler coordinates between the Work Load Manager (WLM) and JES to ensure dependencies for jobs and started tasks are met before allowing them to begin execution. It can also enforce policies related to job and task execution.

(MSTJCL00)

```
VIEW       K2.PARMLIB(MSTJCL00) - 01.00
Command ===>
****** ***************************** Top of Data **
000001 //MSTJCL00 JOB  MSGLEVEL=(1,1),TIME=1440
000002 //          EXEC PGM=IEEMB860,DPRTY=(15,15)
000003 //STCINRDR DD  SYSOUT=(A,INTRDR)
000004 //TSOINRDR DD  SYSOUT=(A,INTRDR)
000005 //IEFPDSI  DD DSN=K2.PROCLIB,DISP=SHR,
000006 //             UNIT=SYSALLDA,VOL=SER=USRVS1
000007 //          DD  DSN=SYS1.PROCLIB,DISP=SHR
000008 //IEFPARM  DD  DSN=SYS1.PARMLIB,DISP=SHR
000009 //SYSUADS  DD DSN=SYS1.VS01.UADS,
000010 //             DISP=SHR
000011 //SYSLBC   DD DSN=SYS1.VS01.BRODCAST,
000012 //             DISP=SHR
****** ***************************** Bottom of Data
```

IPL-ING THE LAB SYSTEM

SYS0.IPLPARM(LOADK2)

How can this JCL do anything if JES hasn't been started yet?

The module lives in SYS1.LINKLIB as an executable. The executable contains DD statements and interprets them to construct SVC99 Supervisor Calls to allocate data sets.

```
MSTJCL00 CSECT
         DC    CL80'//MSTJCL00 JOB MSGLEVEL=(1,1),TIME=1440'
         DC    CL80'//         EXEC PGM=IEEMB860'
         DC    CL80'//STCINRDR DD SYSOUT=(A,INTRDR)'
         DC    CL80'//TSOINRDR DD SYSOUT=(A,INTRDR)'
         DC    CL80'//IEFPDSI  DD DSN=SYS1.PROCLIB,DISP=SHR'
         DC    CL80'//SYSUADS  DD DSN=SYS1.UADS,DISP=SHR'
         DC    CL80'/*'
         END
```

```
VIEW       K2.PARMLIB(MSTJCL00) - 01.00
Command ===>
****** ***************************** Top of Data **
000001 //MSTJCL00 JOB  MSGLEVEL=(1,1),TIME=1440
000002 //         EXEC PGM=IEEMB860,DPRTY=(15,15)
000003 //STCINRDR DD  SYSOUT=(A,INTRDR)
000004 //TSOINRDR DD  SYSOUT=(A,INTRDR)
000005 //IEFPDSI  DD DSN=K2.PROCLIB,DISP=SHR,
000006 //            UNIT=SYSALLDA,VOL=SER=USRVS1
000007 //         DD  DSN=SYS1.PROCLIB,DISP=SHR
000008 //IEFPARM  DD  DSN=SYS1.PARMLIB,DISP=SHR
000009 //SYSUADS  DD DSN=SYS1.VS01.UADS,
000010 //            DISP=SHR
000011 //SYSLBC   DD DSN=SYS1.VS01.BRODCAST,
000012 //            DISP=SHR
****** ***************************** Bottom of Data
```

IPL-ING THE LAB SYSTEM

SYS0.IPLPARM(LOADK2)

SYS1.PARMLIB(IEFSSN00)

(IEFSSN01)

```
BROWSE    SYS1.PARMLIB(IEFSSN00) - 01.01          Line 0000000000 Col
Command ===>                                                 Scroll ==
******************************** Top of Data ***********************
SUBSYS SUBNAME(CNMP) INITRTN(DSI4LSIT)
SUBSYS SUBNAME(SMS)  INITRTN(IGDSSIIN) INITPARM('ID=00,PROMPT=DISPLAY')
SUBSYS SUBNAME(JES2) PRIMARY(YES) START(NO)
SUBSYS SUBNAME(RACF) INITRTN(IRRSSI00) INITPARM('<')
SUBSYS SUBNAME(LOGR) INITRTN(IXGSSINT)
SUBSYS SUBNAME(RRS)
SUBSYS SUBNAME(FFST)
SUBSYS SUBNAME(OMVS)
SUBSYS SUBNAME(ISPF)
SUBSYS SUBNAME(TNF)
SUBSYS SUBNAME(VMCF)
******************************** Bottom of Data ********************
```

How does the Master Scheduler know which subsystems to start? The information is located in members named IEFSSNxx somewhere in the PARMLIB concatenation. Our K2 IPL finds two such members during IPL. The one shown here indicates JES2 is the primary subsystem; there can be only one of those. Several others are also specified. They're known as secondary subsystems.

IPL-ING THE LAB SYSTEM

SYS0.IPLPARM(LOADK2)

SYS1.PARMLIB(IEFSSN00)

(IEFSSN01)

```
BROWSE    SYS1.PARMLIB(IEFSSN01) - 01.00          Line 000000000
Command ===>                                                 Scr
******************************** Top of Data ********************
SUBSYS SUBNAME(CICS) INITRTN(DFHSSIN) INITPARM(DFHSSI00)
SUBSYS SUBNAME(DBD1) INITRTN(DSN3INI) INITPARM('DSN3EPX,-DBD1,S')
SUBSYS SUBNAME(DID1)
SUBSYS SUBNAME(JRLM)
SUBSYS SUBNAME(IRLM)
SUBSYS SUBNAME(CSQ9) INITRTN(CSQ3INI) INITPARM('CSQ3EPX,%CSQ9,M')
****************************** Bottom of Data ******************
```

Member IEFSSN01 in SYS1.PARMLIB contains more specifications for secondary subsystems.

Note that TSO is initiated as a started task, but it is not implemented as a subsystem. It is tightly coupled with z/OS. That's why you don't see it listed in these PARMLIB members.

SETTING UP A FAKE IPL

The next lab involves setting up a fake IPL process. It isn't really possible to replicate some of the most basic steps in the IPL process in a "fake" way. The goal is to gain some insight into how the various PARMLIB members are related to each other.

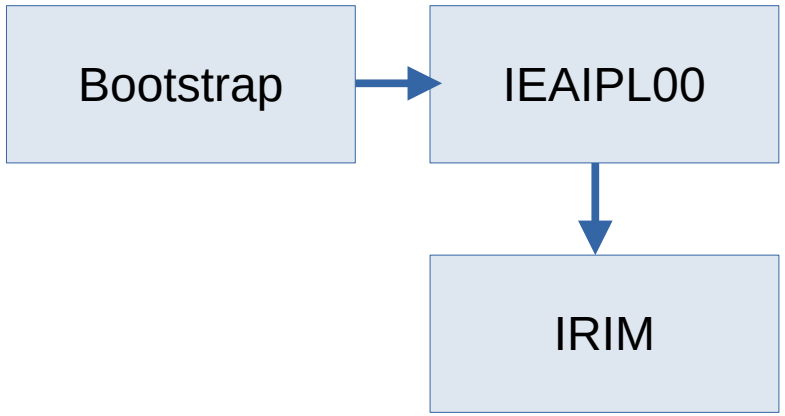
Since we can't actually load an OS into an LPAR in the lab environment, we can't replicate the HMC LOAD option that starts the IPL process. We'll have to assume that was done, and write PARMLIB members and JCL that "pretends" an IPL is happening.

Since the LOAD won't actually happen, the Master Scheduler won't be initiated in the way it is during a real IPL. We'll have to pretend.

A real IPL can't really be tested in a meaningful way other than to give it a try and see what happens, or what doesn't happen. There's no "dry run" option.

Let's walk through the process step by step to be sure we're all clear about what has to happen.

REAL IPL SEQUENCE



Clears central storage (main memory) to zeroes.
Sets up memory areas for the Master Scheduler.
Locates SYS1.NUCLEUS on the specified volume.
Loads a series of IPL Initialization Modules (IRIMs).

Looks for a LOADxx member in:

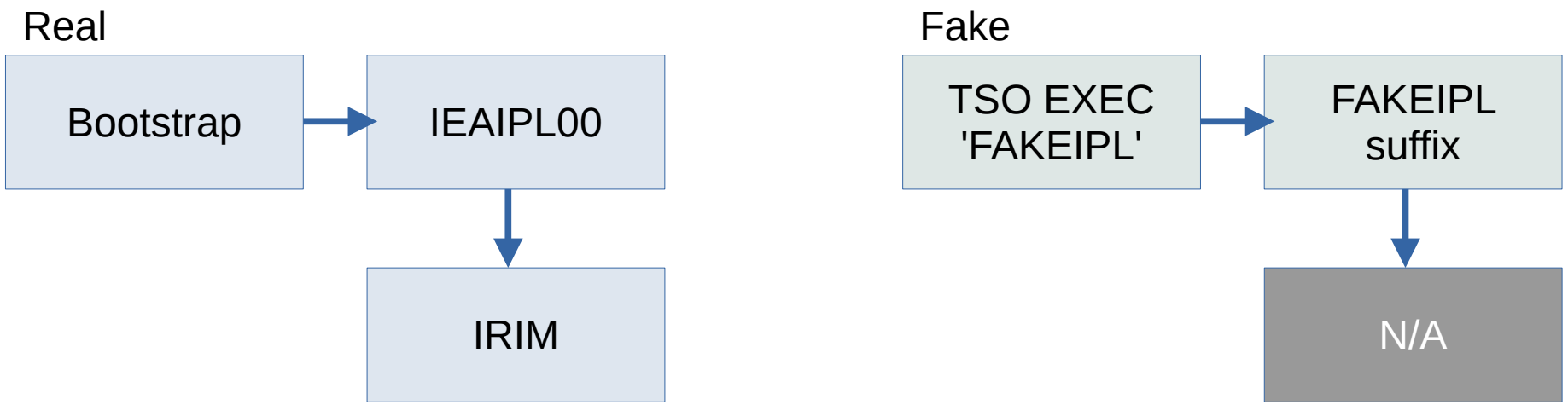
```
SYS0.IPLPARM
SYS1.IPLPARM
  .  .  .
SYS9.IPLPARM
SYS1.PARMLIB
```

If not found, system enters a wait state.

Loads the nucleus, initializes virtual storage for the Master Scheduler, and a bunch of other stuff.

IEAIPL00 is then copied to real memory address zero and stays there.

HOW WILL WE FAKE THIS?

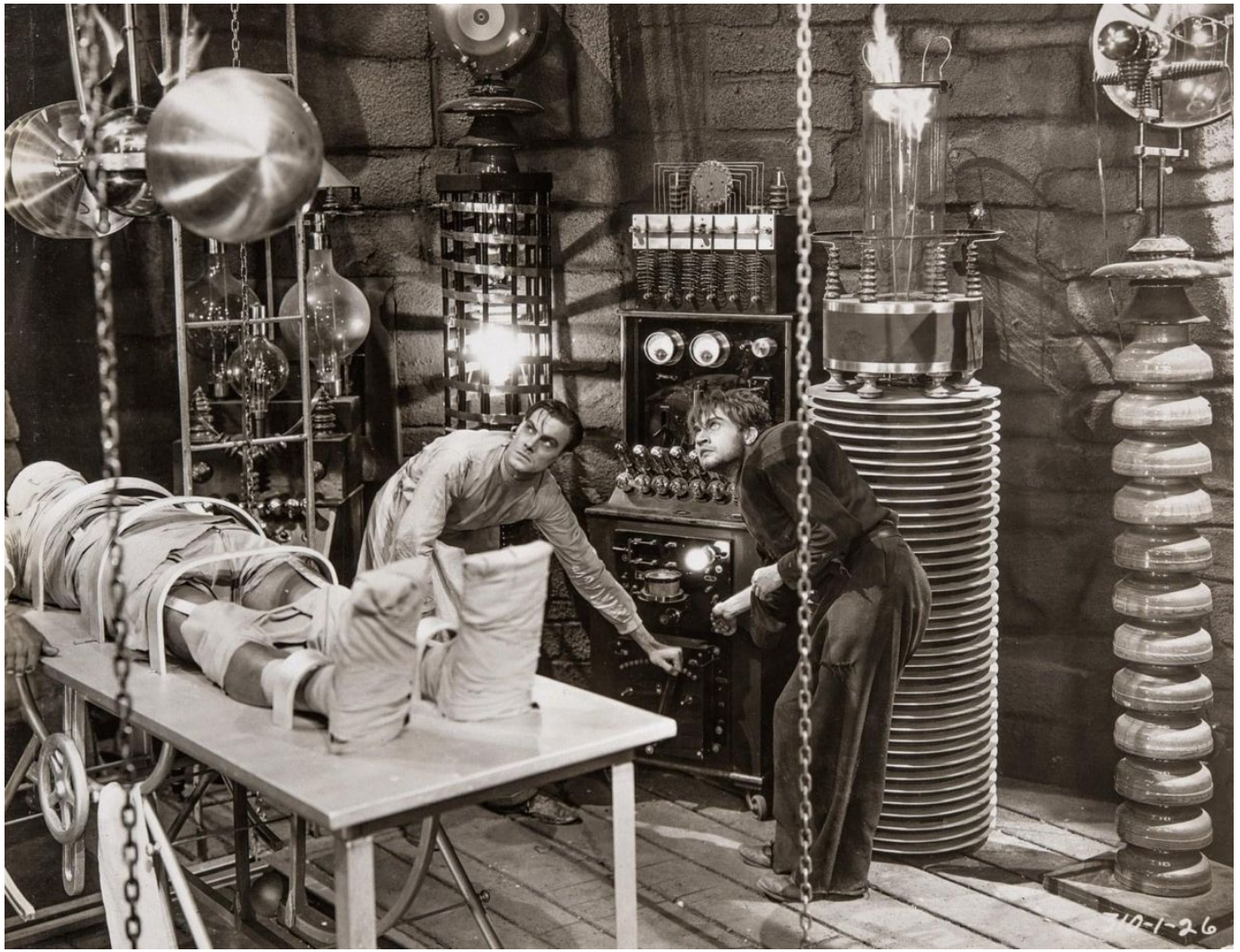


Running FAKEIPL is conceptually equivalent to choosing LOAD on the HMC. You specify a two-character suffix that FAKEIPL will use to search for a LOADxx member in data set FAKE.IPLPARM. Your challenge in the lab is to create these members with a few basic entries:

- **`FAKE.IPLPARM(LOADxx)`**
- **`FAKE.PARMLIB(IEASYMxx)`**
- **`FAKE.PARMLIB(IEASYSxx)`**

When you run FAKEIPL, it will look for those members and the basic entries and if everything looks okay it will start a fake MSTJCL00 job that displays the IPL date and time. Details are in the lab notes.

LAB Z/OS 24: SET UP A FAKE IPL



WHAT YOU KNOW AND WHAT YOU CAN DO

- You understand the general flow of a z/OS IPL process, and some of the key data sets needed for system configuration.

WHAT'S NEXT

- z/OS Environment Design, Configuration Planning, and Maintenance Philosophy